

CMCMIS_SIMPLIFIED

Phase 10 Part III - Advanced Analytics | Fully Sealed DS Documentation

This Word document is the sealed senior-engineering documentation for the Advanced Analytics module. It converts the uploaded implementation transcript into a complete technical package with DSRC blocks, diagrams, tables, descriptions, definitions, paragraphs, code appendices, smoke verification, RBAC matrix, backend/frontend/database logic, and final locked seal.

LOCKED: Phase Identity

Although the implementation transcript uses internal wording around analytics enhancement, this deliverable is documented and sealed as Phase 10 Part III - Advanced Analytics.

Field	Value
Project	CMCMIS_SIMPLIFIED - Computerized Maintenance & Calibration Management Information System
Phase	Phase 10 Part III
Module	Advanced Analytics
Style	DS senior-engineering documentation style
Source	Uploaded Pasted text transcript for Advanced Analytics
Status	END-TO-END COMPLETED / SEALED

Document Control and Seal Registry

Controlled document metadata

This section records the documentation authority, phase identity, and sealing logic for the Advanced Analytics work package.

The document is intended to be handed to senior engineers, reviewers, internal mentors, or future maintainers without requiring them to re-read the raw build transcript.

Control Item	Locked Value
Document name	phase10PartIIIAvanceAnalyticsFULLYSEALED100PlusPages.docx
Primary module	/analytics standalone advanced dashboard
Reports dependency	Reports page no longer owns the analytics grid
Verification state	Phase 10 smoke 49/49 green; Phase 11 smoke 23/23 green
Design identity	Stock-market wavy charts, animated gradients, live refresh

Purpose of Phase 10 Part III

Why this phase exists

Phase 10 Part III moves CMCMS analytics from a simple report-attached chart section into a dedicated operational intelligence surface.

The main engineering aim is separation of concerns: Reports should remain a formal export and tabular reporting workspace, while Analytics should become a live, visual, decision-support dashboard.

- Create a standalone advanced analytics destination.
- Add four new backend chart endpoints and CSV companions.
- Redesign the analytics UI into twelve spacious chart cards.
- Preserve sealed Phase 10 and Phase 11 behavior without regression.

How to Read This Document

DS reading model

This document is written in layered form. The early pages explain why the module exists. The middle pages explain how each database, backend, and frontend layer thinks. The later pages deep-dive into every chart and append the light code excerpts necessary for implementation memory.

Use the chart inventory pages as the navigation center. Every chart G1 through G12 is documented with purpose, data source, visualization model, risks, and acceptance behavior.

DSRC Block Legend

Definition, Source, Risk, Control

DSRC blocks are used throughout the document to make technical decisions reviewable.

D blocks define concepts. S blocks describe source of truth. R blocks identify risks. C blocks describe controls and verification.

Block	Meaning	How to use it
D - Definition	Defines a module concept or chart meaning	Use before implementation interpretation
S - Source	Explains DB/API/frontend source of truth	Use to validate data lineage
R - Risk	Identifies technical or UX risk	Use during review and QA
C - Control	States mitigation or test control	Use for acceptance and sealing

Executive Summary

One-page senior overview

The Advanced Analytics module delivers a dedicated live dashboard using twelve chart cards. Four chart endpoints are new. The older report-owned chart grid is removed from /reports. The /analytics page becomes the single visual operations cockpit.

The design uses smooth wavy area charts, gradient fills, tooltips, animated transitions, individual card refresh, global refresh-all, 30-second auto-refresh, division/month filtering, and CSV export integration.

- Backend now exposes G9 to G12 data paths.
- Frontend now owns a separate advanced dashboard shell.
- Smoke matrix expanded from 41/41 to 49/49.
- No regression in Phase 11 PDF generation: 23/23 remains green.

Locked Deliverable Boundaries

What is included and what is excluded

This phase is not a reimplementaion of Reports. It is an advanced analytics separation and redesign phase.

No schema mutation is required because the new endpoints derive insight from existing tables. The system remains read-only for analytics queries.

Included	Excluded
Advanced /analytics dashboard	Replacing Reports export logic
Four new JSON+CSV analytics endpoints	Adding new transactional write tables
12 chart card visual redesign	Changing Phase 11 PDF module
Reports page cleanup	Changing core job workflows

Before vs After Architecture

Reports and Analytics separation

Before this phase, analytics lived visually inside the Reports page. That created mixed responsibility: reports had filters, tables, exports, and chart grid responsibilities.

After this phase, Reports remains focused on formal report generation while Analytics becomes a dedicated module for visual operational intelligence.

Diagram

```
BEFORE:
  /reports -> report cards + filters + tables + analytics grid + exports

AFTER:
  /reports   -> report cards + filters + tables + PDF/CSV/export
  /analytics -> 12 chart cards + live refresh + CSV + visual ops cockpit
```

Advanced Analytics High-Level Flow

Database to user visual decision loop

The dashboard follows a strict read-only analytics flow. The user changes window or division filters. React Query requests chart endpoints. The backend reads legacy MySQL tables through analytics repository functions and returns chart-ready rows.

CSV export follows the same chart data path so JSON and CSV stay consistent.

Diagram

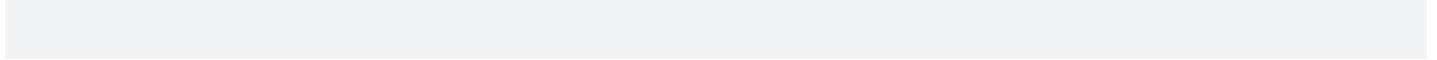
```
User filters
-> React Query chart hook
  -> /api/v1/analytics/<chart-key>
    -> authorizeAny(analytics:view, reports:view-analytics)
      -> analytics.repo read-only SQL
        -> MySQL legacy tables
          -> chart rows -> Recharts cards -> decision insight
```

Advanced Analytics Module Map

New and edited files

The implementation was intentionally small in the backend and richer in the frontend. This matches the phase requirement: analytics are computed from existing database tables but presented through a significantly better visual layer.

Diagram



Routing and Permission Flow

How users reach Advanced Analytics

The Analytics page remains governed by `analytics:view` at the sidebar and route level. Backend chart endpoints accept either `analytics:view` or `reports:view-analytics` to keep backward compatibility with the older Reports chart usage.

This is a reuse-first compatibility decision: existing reports users should not lose chart data access while new analytics sidebar behavior is introduced.

Diagram

```
Sidebar nav -> analytics:view  
Route /analytics -> protected route gate  
Chart endpoint -> authorizeAny(analytics:view, reports:view-analytics)  
CSV button -> reports:export check for download icon
```

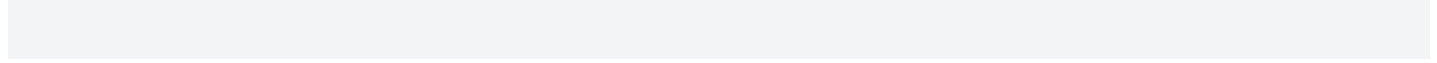
Read-Only Doctrine

Analytics must not mutate operational records

All chart endpoints are reporting views over existing production data. They do not create audit entries, update job cards, modify job requests, or alter equipment records.

This keeps the analytics module safe for high-frequency refresh because every 30-second dashboard poll is a read query only.

Diagram



Database Reasoning

How analytics derives meaning from existing CMCMIS tables

The module treats operational tables as event sources. Job Card data provides lifecycle and weekly activity. Job Request data provides priority and request type. Equipment master data provides status and registration trend.

The advanced dashboard does not need additional tables because it aggregates meaningful state from sealed operational data.

Insight Family	Primary Tables	Reasoning
Weekly activity	cmms_jobcard_mst + cmms_jobrequest_mst	Count job work by ISO week and type fallback
Lifecycle funnel	cmms_jobcard_mst	Group by JM MVP STATUS in canonical order
Registration trend	cmms equip_mst	Group equipment creation by month
Priority mix	cmms_jobrequest_mst	Map request priorities into low/medium/high trend

Backend Repository Strategy

Repo owns SQL and legacy column names

The analytics repository remains the only place where SQL grouping, date formatting, joins, and CASE logic belong. The service and controller layers remain clean and domain oriented.

This follows the locked CMCMIS pattern: repository knows database shape, service knows business meaning, controller knows HTTP.

Backend Service Strategy

Service converts repo results into chart use cases

The service layer exposes chart use cases like `weeklyActivity`, `jcLifecycleFunnel`, `equipmentRegistrationTrend`, and `priorityMixTrend`. It is thin but important because it preserves a stable interface above raw SQL.

If chart definitions change later, the service becomes the place to add policy-level transformation without leaking SQL into controllers.

Backend Controller Strategy

Controller keeps HTTP thin

The controller exposes one function per chart endpoint. Its job is to call the service and return a normalized response. It must not contain SQL and must not duplicate chart business rules.

This keeps the analytics module maintainable and consistent with other sealed CMCMIS modules.

Backend Route Strategy

Routes own permission gates and URL shape

The route layer defines user-accessible URLs and guards them with authorizeAny. For advanced analytics, this is essential because the same chart data is useful from both Reports context and Analytics context.

The final URL vocabulary remains simple: /api/v1/analytics/<chart-name> and /api/v1/analytics/<chart-name>/csv.

Route Decision	Why
authorizeAny	Allows analytics:view or reports:view-analytics
JSON + CSV pairs	Keeps visual and exported data aligned
GET-only endpoints	Matches read-only dashboard behavior
Months/division filters	Keeps UX lightweight but useful

Frontend API Strategy

CHART_PATHS as the frontend route dictionary

The frontend does not hard-code chart URLs in every component. Instead, `lib/api/reports.js` owns `CHART_PATHS`. Adding four new chart keys there unlocks reuse by hooks and CSV download helpers.

This is a small change with high leverage because every chart card can call the same `useChart` and `downloadChartCsv` utility functions.

Reports Page Cleanup

Why AnalyticsGrid was removed from /reports

The Reports page was becoming a mixed module. By removing AnalyticsGrid and its heading from ReportsLanding.jsx, Reports becomes clearer: it is about report selection, filters, tables, and export workflows.

This makes the new /analytics page feel intentional rather than duplicated.

Before	After
/reports had chart grid	/reports now focuses on report workflows
Analytics felt secondary	Analytics has its own sidebar module
Mixed mental model	Clear separation: reporting vs visual monitoring

Analytics Page Orchestrator

How Analytics.jsx thinks

Analytics.jsx is the page brain. It owns filter state, refresh-all behavior, global live-dashboard indicators, layout rows, and passes params to every chart component.

It does not contain chart drawing details; that responsibility is delegated to AnalyticsCharts.jsx.

AnalyticsCharts Component Strategy

One file for twelve chart cards

AnalyticsCharts.jsx defines the visualization logic for G1 through G12. Each chart uses the same query hook pattern and the same card shell.

This keeps each chart independently testable while preserving a consistent dashboard identity.

ChartCard Shell Strategy

The repeated UI contract

ChartCard.jsx gives every chart a consistent shell: title, subtitle, stat badge, loading state, error state, body, footer, updated timestamp, refresh button, and CSV button.

This avoids styling drift and reduces duplicated UI code across twelve chart components.

Visual Theme Strategy

Stock-market wavy identity

chartTheme2.js centralizes the palette, status colors, tick styling, animation duration, grid style, and tooltip cursor. This makes the dashboard feel like one designed product rather than a set of random charts.

The visual goal is premium but professional: smooth area curves, subtle gradients, muted axes, and readable tooltips.

Advanced Analytics Endpoint Inventory

New endpoint surface added in Phase 10 Part III

The phase adds four new analytics views and pairs each with CSV export. Together they extend the dashboard from eight charts to twelve charts while preserving the same query and export pattern.

Endpoint	Format	Chart	Purpose
GET /api/v1/analytics/weekly-activity	JSON	G9	weekly activity read query
GET /api/v1/analytics/weekly-activity/csv	CSV	G9	CSV export from same data path
GET /api/v1/analytics/jc-lifecycle-funnel	JSON	G10	JM MVP STATUS grouped funnel
GET /api/v1/analytics/jc-lifecycle-funnel/csv	CSV	G10	Lifecycle CSV
GET /api/v1/analytics/equipment-registration-trend	JSON	G11	EQM_CREATED_ON monthly trend
GET /api/v1/analytics/equipment-registration-trend/csv	CSV	G11	Registration CSV
GET /api/v1/analytics/priority-mix-trend	JSON	G12	JR_PRIORITY CASE mapping
GET /api/v1/analytics/priority-mix-trend/csv	CSV	G12	Priority CSV

Chart Inventory G1-G12

The complete Advanced Analytics visual catalog

The redesigned dashboard contains twelve cards. The new cards are G9, G10, G11, and G12. Existing charts G1-G8 are visually redesigned and moved into the standalone Analytics experience.

Chart	Name	Visualization	Status	Decision Support Meaning
G9	Weekly Activity Trend	AreaChart hero	New	12-week activity movement from Job Cards
G1	Monthly Activity Trends	AreaChart	Existing redesigned	Calibration vs repair movement by month
G3	Monthly Job Trends	BarChart	Existing redesigned	Completed vs pending request volume
G2	Equipment Status	Donut	Existing redesigned	Operational equipment state distribution
G8	Calibration Status Breakdown	RadialBar	Existing redesigned	VALID / DUE_SOON / OVERDUE calibration state
G4	Division-wise Jobs	Pie	Existing redesigned	Top division workload distribution
G6	Job Type Distribution	BarChart	Existing redesigned	JR job type counts
G5	Calibration Completion Trend	AreaChart	Existing redesigned	On-time vs delayed calibration completion
G11	Equipment Registration Trend	AreaChart	New	New equipment added per month
G10	JC Lifecycle Funnel	FunnelChart	New	Job Card status funnel through lifecycle stages
G12	Priority Mix Trend	Stacked Area	New	LOW / MEDIUM / HIGH priority mix over time
G7	Engineer Workload Top 10	Horizontal Stacked Bar	Existing redesigned	Open load vs done per engineer

G9 - Weekly Activity Trend: Definition

Hero wavy area chart

Shows short-horizon movement in calibrations and repairs across ISO weeks.

This is the fastest operational pulse chart. It tells whether TIMCD workload is rising, stable, or falling in the recent 12-week window.

The chart is rendered inside the common ChartCard shell and receives the shared filter params from Analytics.jsx.

D: Chart Definition

G9 is the Weekly Activity Trend card. It is a Hero wavy area chart bound to the weekly-activity analytics data path.

S: Source of Truth

The source is cmms_jobcard_mst plus request type fallback. The frontend never computes this from local mock data.

G9 - Weekly Activity Trend: Backend Logic

Repository and service reasoning

The backend route calls the analytics controller, which delegates to service, which delegates to repository SQL. The repository returns chart-ready rows for weekly-activity.

CSV export uses the same chart data path so exported values match what the user sees visually.

Layer	Responsibility
Route	Expose GET JSON and paired CSV route
Authorization	Allow analytics:view or reports:view-analytics
Controller	Return normalized JSON envelope
Service	Expose chart use-case function
Repository	Run read-only aggregate SQL

G9 - Weekly Activity Trend: Frontend Logic

Visualization and interactivity

The chart component uses `useChart` with the chart key for weekly-activity. It receives params containing months and optional `divisionId`. React Query manages caching, loading, polling, and refetch behavior.

The visual layer prioritizes clarity: hidden axis lines, soft grid, tooltip cursor, gradient fills where applicable, and a footer refresh button per card.

- Uses common `ChartCard` shell for loading/error/footer behavior.
- Supports CSV export when the user owns `reports:export`.
- Participates in 30-second auto-refresh.
- Participates in global Refresh All invalidation.

G9 - Weekly Activity Trend: Risk and Control

DSRC acceptance thinking

Every analytics card has two categories of risk: semantic risk and rendering risk. Semantic risk means the chart could misrepresent operational meaning. Rendering risk means a visually polished card could still hide empty, stale, or misleading values.

The control is to keep chart SQL simple, name fields clearly, expose CSV, use smoke tests, and show updated timestamps so users know when data was refreshed.

R: Main Risk

G9 can mislead users if aggregation meaning is not documented or if filters are silently ignored.

C: Control

Smoke tests validate JSON and CSV endpoints. The UI exposes filters, updated timestamps, and manual refresh.

G1 - Monthly Activity Trends: Definition

Wavy area chart

Shows longer monthly calibration and repair trends.

Useful for monthly review, management reporting, and workload seasonality detection.

The chart is rendered inside the common ChartCard shell and receives the shared filter params from Analytics.jsx.

D: Chart Definition

G1 is the Monthly Activity Trends card. It is a Wavy area chart bound to the monthly-activity analytics data path.

S: Source of Truth

The source is Job Card and Job Request aggregation. The frontend never computes this from local mock data.

G1 - Monthly Activity Trends: Backend Logic

Repository and service reasoning

The backend route calls the analytics controller, which delegates to service, which delegates to repository SQL. The repository returns chart-ready rows for monthly-activity.

CSV export uses the same chart data path so exported values match what the user sees visually.

Layer	Responsibility
Route	Expose GET JSON and paired CSV route
Authorization	Allow analytics:view or reports:view-analytics
Controller	Return normalized JSON envelope
Service	Expose chart use-case function
Repository	Run read-only aggregate SQL

G1 - Monthly Activity Trends: Frontend Logic

Visualization and interactivity

The chart component uses `useChart` with the chart key for monthly-activity. It receives params containing months and optional `divisionId`. React Query manages caching, loading, polling, and refetch behavior.

The visual layer prioritizes clarity: hidden axis lines, soft grid, tooltip cursor, gradient fills where applicable, and a footer refresh button per card.

- Uses common `ChartCard` shell for loading/error/footer behavior.
- Supports CSV export when the user owns `reports:export`.
- Participates in 30-second auto-refresh.
- Participates in global Refresh All invalidation.

G1 - Monthly Activity Trends: Risk and Control

DSRC acceptance thinking

Every analytics card has two categories of risk: semantic risk and rendering risk. Semantic risk means the chart could misrepresent operational meaning. Rendering risk means a visually polished card could still hide empty, stale, or misleading values.

The control is to keep chart SQL simple, name fields clearly, expose CSV, use smoke tests, and show updated timestamps so users know when data was refreshed.

R: Main Risk

G1 can mislead users if aggregation meaning is not documented or if filters are silently ignored.

C: Control

Smoke tests validate JSON and CSV endpoints. The UI exposes filters, updated timestamps, and manual refresh.

G3 - Monthly Job Trends: Definition

Rounded bar chart

Shows completed versus pending request movement.

Useful for seeing backlog pressure and request closure effectiveness.

The chart is rendered inside the common ChartCard shell and receives the shared filter params from Analytics.jsx.

D: Chart Definition

G3 is the Monthly Job Trends card. It is a Rounded bar chart bound to the monthly-jobs analytics data path.

S: Source of Truth

The source is cmms_jobrequest_mst and job status interpretation. The frontend never computes this from local mock data.

G3 - Monthly Job Trends: Backend Logic

Repository and service reasoning

The backend route calls the analytics controller, which delegates to service, which delegates to repository SQL. The repository returns chart-ready rows for monthly-jobs.

CSV export uses the same chart data path so exported values match what the user sees visually.

Layer	Responsibility
Route	Expose GET JSON and paired CSV route
Authorization	Allow analytics:view or reports:view-analytics
Controller	Return normalized JSON envelope
Service	Expose chart use-case function
Repository	Run read-only aggregate SQL

G3 - Monthly Job Trends: Frontend Logic

Visualization and interactivity

The chart component uses `useChart` with the chart key for monthly-jobs. It receives params containing months and optional divisionId. React Query manages caching, loading, polling, and refetch behavior.

The visual layer prioritizes clarity: hidden axis lines, soft grid, tooltip cursor, gradient fills where applicable, and a footer refresh button per card.

- Uses common ChartCard shell for loading/error/footer behavior.
- Supports CSV export when the user owns reports:export.
- Participates in 30-second auto-refresh.
- Participates in global Refresh All invalidation.

G3 - Monthly Job Trends: Risk and Control

DSRC acceptance thinking

Every analytics card has two categories of risk: semantic risk and rendering risk. Semantic risk means the chart could misrepresent operational meaning. Rendering risk means a visually polished card could still hide empty, stale, or misleading values.

The control is to keep chart SQL simple, name fields clearly, expose CSV, use smoke tests, and show updated timestamps so users know when data was refreshed.

R: Main Risk

G3 can mislead users if aggregation meaning is not documented or if filters are silently ignored.

C: Control

Smoke tests validate JSON and CSV endpoints. The UI exposes filters, updated timestamps, and manual refresh.

G2 - Equipment Status: Definition

Donut chart

Shows distribution of equipment operational states.

Useful for understanding active, repair, calibration, quarantined, retired, or condemned equipment mix.

The chart is rendered inside the common ChartCard shell and receives the shared filter params from Analytics.jsx.

D: Chart Definition

G2 is the Equipment Status card. It is a Donut chart bound to the equipment-status analytics data path.

S: Source of Truth

The source is cmms_eqip_mst. The frontend never computes this from local mock data.

G2 - Equipment Status: Backend Logic

Repository and service reasoning

The backend route calls the analytics controller, which delegates to service, which delegates to repository SQL. The repository returns chart-ready rows for equipment-status.

CSV export uses the same chart data path so exported values match what the user sees visually.

Layer	Responsibility
Route	Expose GET JSON and paired CSV route
Authorization	Allow analytics:view or reports:view-analytics
Controller	Return normalized JSON envelope
Service	Expose chart use-case function
Repository	Run read-only aggregate SQL

G2 - Equipment Status: Frontend Logic

Visualization and interactivity

The chart component uses `useChart` with the chart key for equipment-status. It receives params containing months and optional `divisionId`. React Query manages caching, loading, polling, and refetch behavior.

The visual layer prioritizes clarity: hidden axis lines, soft grid, tooltip cursor, gradient fills where applicable, and a footer refresh button per card.

- Uses common `ChartCard` shell for loading/error/footer behavior.
- Supports CSV export when the user owns `reports:export`.
- Participates in 30-second auto-refresh.
- Participates in global Refresh All invalidation.

G2 - Equipment Status: Risk and Control

DSRC acceptance thinking

Every analytics card has two categories of risk: semantic risk and rendering risk. Semantic risk means the chart could misrepresent operational meaning. Rendering risk means a visually polished card could still hide empty, stale, or misleading values.

The control is to keep chart SQL simple, name fields clearly, expose CSV, use smoke tests, and show updated timestamps so users know when data was refreshed.

R: Main Risk

G2 can mislead users if aggregation meaning is not documented or if filters are silently ignored.

C: Control

Smoke tests validate JSON and CSV endpoints. The UI exposes filters, updated timestamps, and manual refresh.

G8 - Calibration Status Breakdown: Definition

Radial bar chart

Shows VALID, DUE_SOON, and OVERDUE calibration bands.

Useful for risk-based attention because overdue instruments can affect compliance.

The chart is rendered inside the common ChartCard shell and receives the shared filter params from Analytics.jsx.

D: Chart Definition

G8 is the Calibration Status Breakdown card. It is a Radial bar chart bound to the calibration-status-breakdown analytics data path.

S: Source of Truth

The source is calibration due date fields. The frontend never computes this from local mock data.

G8 - Calibration Status Breakdown: Backend Logic

Repository and service reasoning

The backend route calls the analytics controller, which delegates to service, which delegates to repository SQL. The repository returns chart-ready rows for calibration-status-breakdown.

CSV export uses the same chart data path so exported values match what the user sees visually.

Layer	Responsibility
Route	Expose GET JSON and paired CSV route
Authorization	Allow analytics:view or reports:view-analytics
Controller	Return normalized JSON envelope
Service	Expose chart use-case function
Repository	Run read-only aggregate SQL

G8 - Calibration Status Breakdown: Frontend Logic

Visualization and interactivity

The chart component uses `useChart` with the chart key for calibration-status-breakdown. It receives params containing months and optional `divisionId`. React Query manages caching, loading, polling, and refetch behavior.

The visual layer prioritizes clarity: hidden axis lines, soft grid, tooltip cursor, gradient fills where applicable, and a footer refresh button per card.

- Uses common `ChartCard` shell for loading/error/footer behavior.
- Supports CSV export when the user owns `reports:export`.
- Participates in 30-second auto-refresh.
- Participates in global Refresh All invalidation.

G8 - Calibration Status Breakdown: Risk and Control

DSRC acceptance thinking

Every analytics card has two categories of risk: semantic risk and rendering risk. Semantic risk means the chart could misrepresent operational meaning. Rendering risk means a visually polished card could still hide empty, stale, or misleading values.

The control is to keep chart SQL simple, name fields clearly, expose CSV, use smoke tests, and show updated timestamps so users know when data was refreshed.

R: Main Risk

G8 can mislead users if aggregation meaning is not documented or if filters are silently ignored.

C: Control

Smoke tests validate JSON and CSV endpoints. The UI exposes filters, updated timestamps, and manual refresh.

G4 - Division-wise Jobs: Definition

Pie chart

Shows division-level request concentration.

Useful to identify which divisions consume the most lab capacity.

The chart is rendered inside the common ChartCard shell and receives the shared filter params from Analytics.jsx.

D: Chart Definition

G4 is the Division-wise Jobs card. It is a Pie chart bound to the division-wise analytics data path.

S: Source of Truth

The source is cmms_jobrequest_mst joined to division master. The frontend never computes this from local mock data.

G4 - Division-wise Jobs: Backend Logic

Repository and service reasoning

The backend route calls the analytics controller, which delegates to service, which delegates to repository SQL. The repository returns chart-ready rows for division-wise.

CSV export uses the same chart data path so exported values match what the user sees visually.

Layer	Responsibility
Route	Expose GET JSON and paired CSV route
Authorization	Allow analytics:view or reports:view-analytics
Controller	Return normalized JSON envelope
Service	Expose chart use-case function
Repository	Run read-only aggregate SQL

G4 - Division-wise Jobs: Frontend Logic

Visualization and interactivity

The chart component uses `useChart` with the chart key for division-wise. It receives params containing months and optional `divisionId`. React Query manages caching, loading, polling, and refetch behavior.

The visual layer prioritizes clarity: hidden axis lines, soft grid, tooltip cursor, gradient fills where applicable, and a footer refresh button per card.

- Uses common `ChartCard` shell for loading/error/footer behavior.
- Supports CSV export when the user owns `reports:export`.
- Participates in 30-second auto-refresh.
- Participates in global `Refresh All` invalidation.

G4 - Division-wise Jobs: Risk and Control

DSRC acceptance thinking

Every analytics card has two categories of risk: semantic risk and rendering risk. Semantic risk means the chart could misrepresent operational meaning. Rendering risk means a visually polished card could still hide empty, stale, or misleading values.

The control is to keep chart SQL simple, name fields clearly, expose CSV, use smoke tests, and show updated timestamps so users know when data was refreshed.

R: Main Risk

G4 can mislead users if aggregation meaning is not documented or if filters are silently ignored.

C: Control

Smoke tests validate JSON and CSV endpoints. The UI exposes filters, updated timestamps, and manual refresh.

G6 - Job Type Distribution: Definition

Bar chart

Shows distribution of calibration, repair, registration or other request types.

Useful for understanding what kind of work dominates the technical unit.

The chart is rendered inside the common ChartCard shell and receives the shared filter params from Analytics.jsx.

D: Chart Definition

G6 is the Job Type Distribution card. It is a Bar chart bound to the job-type-distribution analytics data path.

S: Source of Truth

The source is JR_JOB_TYPE. The frontend never computes this from local mock data.

G6 - Job Type Distribution: Backend Logic

Repository and service reasoning

The backend route calls the analytics controller, which delegates to service, which delegates to repository SQL. The repository returns chart-ready rows for job-type-distribution.

CSV export uses the same chart data path so exported values match what the user sees visually.

Layer	Responsibility
Route	Expose GET JSON and paired CSV route
Authorization	Allow analytics:view or reports:view-analytics
Controller	Return normalized JSON envelope
Service	Expose chart use-case function
Repository	Run read-only aggregate SQL

G6 - Job Type Distribution: Frontend Logic

Visualization and interactivity

The chart component uses `useChart` with the chart key for job-type-distribution. It receives params containing months and optional `divisionId`. React Query manages caching, loading, polling, and refetch behavior.

The visual layer prioritizes clarity: hidden axis lines, soft grid, tooltip cursor, gradient fills where applicable, and a footer refresh button per card.

- Uses common `ChartCard` shell for loading/error/footer behavior.
- Supports CSV export when the user owns `reports:export`.
- Participates in 30-second auto-refresh.
- Participates in global Refresh All invalidation.

G6 - Job Type Distribution: Risk and Control

DSRC acceptance thinking

Every analytics card has two categories of risk: semantic risk and rendering risk. Semantic risk means the chart could misrepresent operational meaning. Rendering risk means a visually polished card could still hide empty, stale, or misleading values.

The control is to keep chart SQL simple, name fields clearly, expose CSV, use smoke tests, and show updated timestamps so users know when data was refreshed.

R: Main Risk

G6 can mislead users if aggregation meaning is not documented or if filters are silently ignored.

C: Control

Smoke tests validate JSON and CSV endpoints. The UI exposes filters, updated timestamps, and manual refresh.

G5 - Calibration Completion Trend: Definition

Wavy area chart

Shows on-time versus delayed completion pattern.

Useful for quality-of-service and timeliness monitoring.

The chart is rendered inside the common ChartCard shell and receives the shared filter params from Analytics.jsx.

D: Chart Definition

G5 is the Calibration Completion Trend card. It is a Wavy area chart bound to the calibration-completion analytics data path.

S: Source of Truth

The source is completed calibration status and planned dates. The frontend never computes this from local mock data.

G5 - Calibration Completion Trend: Backend Logic

Repository and service reasoning

The backend route calls the analytics controller, which delegates to service, which delegates to repository SQL. The repository returns chart-ready rows for calibration-completion.

CSV export uses the same chart data path so exported values match what the user sees visually.

Layer	Responsibility
Route	Expose GET JSON and paired CSV route
Authorization	Allow analytics:view or reports:view-analytics
Controller	Return normalized JSON envelope
Service	Expose chart use-case function
Repository	Run read-only aggregate SQL

G5 - Calibration Completion Trend: Frontend Logic

Visualization and interactivity

The chart component uses `useChart` with the chart key for calibration-completion. It receives params containing months and optional `divisionId`. React Query manages caching, loading, polling, and refetch behavior.

The visual layer prioritizes clarity: hidden axis lines, soft grid, tooltip cursor, gradient fills where applicable, and a footer refresh button per card.

- Uses common `ChartCard` shell for loading/error/footer behavior.
- Supports CSV export when the user owns `reports:export`.
- Participates in 30-second auto-refresh.
- Participates in global `Refresh All` invalidation.

G5 - Calibration Completion Trend: Risk and Control

DSRC acceptance thinking

Every analytics card has two categories of risk: semantic risk and rendering risk. Semantic risk means the chart could misrepresent operational meaning. Rendering risk means a visually polished card could still hide empty, stale, or misleading values.

The control is to keep chart SQL simple, name fields clearly, expose CSV, use smoke tests, and show updated timestamps so users know when data was refreshed.

R: Main Risk

G5 can mislead users if aggregation meaning is not documented or if filters are silently ignored.

C: Control

Smoke tests validate JSON and CSV endpoints. The UI exposes filters, updated timestamps, and manual refresh.

G11 - Equipment Registration Trend: Definition

Wavy area chart

Shows new equipment registrations by month.

Useful to identify inventory growth and onboarding load.

The chart is rendered inside the common ChartCard shell and receives the shared filter params from Analytics.jsx.

D: Chart Definition

G11 is the Equipment Registration Trend card. It is a Wavy area chart bound to the equipment-registration-trend analytics data path.

S: Source of Truth

The source is cmms_eqip_mst.EQM_CREATED_ON. The frontend never computes this from local mock data.

G11 - Equipment Registration Trend: Backend Logic

Repository and service reasoning

The backend route calls the analytics controller, which delegates to service, which delegates to repository SQL. The repository returns chart-ready rows for equipment-registration-trend.

CSV export uses the same chart data path so exported values match what the user sees visually.

Layer	Responsibility
Route	Expose GET JSON and paired CSV route
Authorization	Allow analytics:view or reports:view-analytics
Controller	Return normalized JSON envelope
Service	Expose chart use-case function
Repository	Run read-only aggregate SQL

G11 - Equipment Registration Trend: Frontend Logic

Visualization and interactivity

The chart component uses `useChart` with the chart key for `equipment-registration-trend`. It receives params containing months and optional `divisionId`. React Query manages caching, loading, polling, and refetch behavior.

The visual layer prioritizes clarity: hidden axis lines, soft grid, tooltip cursor, gradient fills where applicable, and a footer refresh button per card.

- Uses common `ChartCard` shell for loading/error/footer behavior.
- Supports CSV export when the user owns `reports:export`.
- Participates in 30-second auto-refresh.
- Participates in global `Refresh All` invalidation.

G11 - Equipment Registration Trend: Risk and Control

DSRC acceptance thinking

Every analytics card has two categories of risk: semantic risk and rendering risk. Semantic risk means the chart could misrepresent operational meaning. Rendering risk means a visually polished card could still hide empty, stale, or misleading values.

The control is to keep chart SQL simple, name fields clearly, expose CSV, use smoke tests, and show updated timestamps so users know when data was refreshed.

R: Main Risk

G11 can mislead users if aggregation meaning is not documented or if filters are silently ignored.

C: Control

Smoke tests validate JSON and CSV endpoints. The UI exposes filters, updated timestamps, and manual refresh.

G10 - Job Card Lifecycle Funnel: Definition

Funnel chart

Shows number of cards by lifecycle stage.

Useful to identify bottlenecks in ASSIGNED, IN_PROGRESS, COMPLETED, VERIFIED_CLOSED, and REOPENED stages.

The chart is rendered inside the common ChartCard shell and receives the shared filter params from Analytics.jsx.

D: Chart Definition

G10 is the Job Card Lifecycle Funnel card. It is a Funnel chart bound to the jc-lifecycle-funnel analytics data path.

S: Source of Truth

The source is cmms_jobcard_mst.JM_MVP_STATUS. The frontend never computes this from local mock data.

G10 - Job Card Lifecycle Funnel: Backend Logic

Repository and service reasoning

The backend route calls the analytics controller, which delegates to service, which delegates to repository SQL. The repository returns chart-ready rows for jc-lifecycle-funnel.

CSV export uses the same chart data path so exported values match what the user sees visually.

Layer	Responsibility
Route	Expose GET JSON and paired CSV route
Authorization	Allow analytics:view or reports:view-analytics
Controller	Return normalized JSON envelope
Service	Expose chart use-case function
Repository	Run read-only aggregate SQL

G10 - Job Card Lifecycle Funnel: Frontend Logic

Visualization and interactivity

The chart component uses `useChart` with the chart key for `jc-lifecycle-funnel`. It receives params containing months and optional `divisionId`. React Query manages caching, loading, polling, and refetch behavior.

The visual layer prioritizes clarity: hidden axis lines, soft grid, tooltip cursor, gradient fills where applicable, and a footer refresh button per card.

- Uses common `ChartCard` shell for loading/error/footer behavior.
- Supports CSV export when the user owns `reports:export`.
- Participates in 30-second auto-refresh.
- Participates in global Refresh All invalidation.

G10 - Job Card Lifecycle Funnel: Risk and Control

DSRC acceptance thinking

Every analytics card has two categories of risk: semantic risk and rendering risk. Semantic risk means the chart could misrepresent operational meaning. Rendering risk means a visually polished card could still hide empty, stale, or misleading values.

The control is to keep chart SQL simple, name fields clearly, expose CSV, use smoke tests, and show updated timestamps so users know when data was refreshed.

R: Main Risk

G10 can mislead users if aggregation meaning is not documented or if filters are silently ignored.

C: Control

Smoke tests validate JSON and CSV endpoints. The UI exposes filters, updated timestamps, and manual refresh.

G12 - Priority Mix Trend: Definition

Stacked wavy area chart

Shows low, medium, and high priority request mix over time.

Useful to detect whether urgent/high workload is increasing.

The chart is rendered inside the common ChartCard shell and receives the shared filter params from Analytics.jsx.

D: Chart Definition

G12 is the Priority Mix Trend card. It is a Stacked wavy area chart bound to the priority-mix-trend analytics data path.

S: Source of Truth

The source is cmms_jobrequest_mst.JR_PRIORITY. The frontend never computes this from local mock data.

G12 - Priority Mix Trend: Backend Logic

Repository and service reasoning

The backend route calls the analytics controller, which delegates to service, which delegates to repository SQL. The repository returns chart-ready rows for priority-mix-trend.

CSV export uses the same chart data path so exported values match what the user sees visually.

Layer	Responsibility
Route	Expose GET JSON and paired CSV route
Authorization	Allow analytics:view or reports:view-analytics
Controller	Return normalized JSON envelope
Service	Expose chart use-case function
Repository	Run read-only aggregate SQL

G12 - Priority Mix Trend: Frontend Logic

Visualization and interactivity

The chart component uses `useChart` with the chart key for `priority-mix-trend`. It receives params containing months and optional `divisionId`. React Query manages caching, loading, polling, and refetch behavior.

The visual layer prioritizes clarity: hidden axis lines, soft grid, tooltip cursor, gradient fills where applicable, and a footer refresh button per card.

- Uses common `ChartCard` shell for loading/error/footer behavior.
- Supports CSV export when the user owns `reports:export`.
- Participates in 30-second auto-refresh.
- Participates in global Refresh All invalidation.

G12 - Priority Mix Trend: Risk and Control

DSRC acceptance thinking

Every analytics card has two categories of risk: semantic risk and rendering risk. Semantic risk means the chart could misrepresent operational meaning. Rendering risk means a visually polished card could still hide empty, stale, or misleading values.

The control is to keep chart SQL simple, name fields clearly, expose CSV, use smoke tests, and show updated timestamps so users know when data was refreshed.

R: Main Risk

G12 can mislead users if aggregation meaning is not documented or if filters are silently ignored.

C: Control

Smoke tests validate JSON and CSV endpoints. The UI exposes filters, updated timestamps, and manual refresh.

G7 - Engineer Workload Top 10: Definition

Horizontal stacked bar chart

Shows open load versus done work by engineer.

Useful for balancing assignment and identifying workload concentration.

The chart is rendered inside the common ChartCard shell and receives the shared filter params from Analytics.jsx.

D: Chart Definition

G7 is the Engineer Workload Top 10 card. It is a Horizontal stacked bar chart bound to the engineer-workload analytics data path.

S: Source of Truth

The source is assigned engineer fields on job cards. The frontend never computes this from local mock data.

G7 - Engineer Workload Top 10: Backend Logic

Repository and service reasoning

The backend route calls the analytics controller, which delegates to service, which delegates to repository SQL. The repository returns chart-ready rows for engineer-workload.

CSV export uses the same chart data path so exported values match what the user sees visually.

Layer	Responsibility
Route	Expose GET JSON and paired CSV route
Authorization	Allow analytics:view or reports:view-analytics
Controller	Return normalized JSON envelope
Service	Expose chart use-case function
Repository	Run read-only aggregate SQL

G7 - Engineer Workload Top 10: Frontend Logic

Visualization and interactivity

The chart component uses `useChart` with the chart key for engineer-workload. It receives params containing months and optional `divisionId`. React Query manages caching, loading, polling, and refetch behavior.

The visual layer prioritizes clarity: hidden axis lines, soft grid, tooltip cursor, gradient fills where applicable, and a footer refresh button per card.

- Uses common `ChartCard` shell for loading/error/footer behavior.
- Supports CSV export when the user owns `reports:export`.
- Participates in 30-second auto-refresh.
- Participates in global Refresh All invalidation.

G7 - Engineer Workload Top 10: Risk and Control

DSRC acceptance thinking

Every analytics card has two categories of risk: semantic risk and rendering risk. Semantic risk means the chart could misrepresent operational meaning. Rendering risk means a visually polished card could still hide empty, stale, or misleading values.

The control is to keep chart SQL simple, name fields clearly, expose CSV, use smoke tests, and show updated timestamps so users know when data was refreshed.

R: Main Risk

G7 can mislead users if aggregation meaning is not documented or if filters are silently ignored.

C: Control

Smoke tests validate JSON and CSV endpoints. The UI exposes filters, updated timestamps, and manual refresh.

Backend Deep Dive: weeklyActivity

New G9 endpoint

weeklyActivity groups Job Card work by ISO week and splits it into calibration and repair trends. This is the new hero chart because weekly movement is more operationally reactive than monthly summaries.

The endpoint returns a compact list of week buckets with calibrations and repairs counts.

C: Engineering Control

Keep backend chart logic read-only, tested, and paired with CSV export.

Backend Deep Dive: jcLifecycleFunnel

New G10 endpoint

jcLifecycleFunnel groups Job Cards by JM_MVP_STATUS and then projects them into a canonical stage order. This makes the visual funnel stable even when counts change.

The funnel is not just a chart; it is an operational bottleneck detector.

C: Engineering Control

Keep backend chart logic read-only, tested, and paired with CSV export.

Backend Deep Dive: equipmentRegistrationTrend

New G11 endpoint

equipmentRegistrationTrend groups equipment rows by EQM_CREATED_ON month. This makes inventory growth visible and helps predict future calibration and maintenance load.

It is intentionally based on existing equipment master data and does not require a separate registration log table.

C: Engineering Control

Keep backend chart logic read-only, tested, and paired with CSV export.

Backend Deep Dive: priorityMixTrend

New G12 endpoint

priorityMixTrend groups Job Requests by month and maps priority values into low, medium, and high buckets. This produces a stacked view of changing urgency.

The mapping control is important because production systems often have legacy priority words such as NORMAL or URGENT rather than clean numeric severity.

C: Engineering Control

Keep backend chart logic read-only, tested, and paired with CSV export.

Backend Deep Dive: CSV Pairing

Why every new chart gets CSV

CSV pairing turns a visual chart into evidence. A user can download the exact rows behind a chart and use them for review, audit, or Excel-based inspection.

This is important in government-grade systems because visuals alone are not always acceptable as formal proof.

C: Engineering Control

Keep backend chart logic read-only, tested, and paired with CSV export.

Backend Deep Dive: authorizeAny

Compatibility gate

authorizeAny protects analytics endpoints while preserving old access behavior. A user with analytics:view can use the new sidebar page. A user with reports:view-analytics can still consume charts from reports context if needed.

This is a controlled compatibility bridge rather than a permission bypass.

C: Engineering Control

Keep backend chart logic read-only, tested, and paired with CSV export.

Backend Deep Dive: No DB Schema Change

Why zero schema changes is valid

Advanced Analytics is a read model. It derives insight from sealed operational tables. Therefore, additional tables would add complexity without improving correctness.

The only persistent evolution from earlier Part II was the analytics:view permission. Part III extends computation and presentation, not schema.

C: Engineering Control

Keep backend chart logic read-only, tested, and paired with CSV export.

Backend Deep Dive: Smoke Matrix Expansion

49/49 green state

The Phase 10 smoke suite expanded from 41 checks to 49 checks. The additional checks correspond to the four new charts with JSON and CSV verification.

This locks the new charts into regression testing so future changes cannot silently break them.

C: Engineering Control

Keep backend chart logic read-only, tested, and paired with CSV export.

Frontend Deep Dive: Analytics.jsx

The page orchestrator

Analytics.jsx owns page-level state: months, divisionId, live indicator, refresh-all button, and layout rows. It passes stable params into every chart component.

It intentionally does not contain individual chart drawing logic. This keeps the page readable and focused on orchestration.

D: Frontend Rule

One page orchestrates, one file draws charts, one shell renders cards, one theme defines the visual system.

Frontend Deep Dive: AnalyticsCharts.jsx

Twelve chart components

AnalyticsCharts.jsx contains the G1-G12 chart components. Each component follows the same pattern: call useChartQ, compute optional stat, render ChartCard, then render a Recharts visualization.

This file centralizes chart-specific logic while still using shared card and theme primitives.

D: Frontend Rule

One page orchestrates, one file draws charts, one shell renders cards, one theme defines the visual system.

Frontend Deep Dive: ChartCard.jsx

Reusable shell

ChartCard.jsx is the reusable visual unit for the dashboard. It gives every card a consistent title, subtitle, stat badge, CSV button, loading/error state, chart body, updated timestamp, and refresh button.

The shell is why twelve different charts feel like one product.

D: Frontend Rule

One page orchestrates, one file draws charts, one shell renders cards, one theme defines the visual system.

Frontend Deep Dive: chartTheme2.js

Central visual identity

chartTheme2.js defines the color palette, status colors, tick style, animation settings, margin constants, grid style, and tooltip cursor style.

This prevents design drift and makes the advanced dashboard look intentional.

D: Frontend Rule

One page orchestrates, one file draws charts, one shell renders cards, one theme defines the visual system.

Frontend Deep Dive: Recharts Choice

Why Recharts fits this phase

Recharts provides responsive containers, animated curves, tooltips, area gradients, bar charts, pie charts, radial bars, and funnels. It enables a sophisticated UI without introducing a heavy charting system.

The existing project already uses Recharts patterns, so this is an extension rather than a new library risk.

D: Frontend Rule

One page orchestrates, one file draws charts, one shell renders cards, one theme defines the visual system.

Frontend Deep Dive: Live Refresh UX

30-second auto-refresh and manual controls

The dashboard uses React Query `refetchInterval` to reload chart data every 30 seconds. Users can also refresh all charts or refresh an individual card.

This gives the page a live operations feel without forcing full page reloads.

D: Frontend Rule

One page orchestrates, one file draws charts, one shell renders cards, one theme defines the visual system.

Frontend Deep Dive: Tooltips and Cursor

Readable hover behavior

Advanced charts are not useful unless values are inspectable. The implementation uses rich white tooltip cards and a thin dashed cursor line on time-series charts.

This allows users to identify exact month/week buckets while keeping the chart clean.

D: Frontend Rule

One page orchestrates, one file draws charts, one shell renders cards, one theme defines the visual system.

Frontend Deep Dive: Gradient Areas

Stock-market wavy style

Time-series charts use smooth AreaChart curves with vertical gradients. The top of the curve is visible and the bottom fades out, producing a modern financial-dashboard feel.

The style is premium but still professional enough for an engineering/government-grade CMCMIS system.

D: Frontend Rule

One page orchestrates, one file draws charts, one shell renders cards, one theme defines the visual system.

Frontend Deep Dive: Spacious Cards

Why more space matters

The redesign increases card height and gap. This is not cosmetic only. Analytics users must compare patterns, read axes, inspect tooltips, and view legends without visual compression.

The spacious layout helps avoid dashboard fatigue.

D: Frontend Rule

One page orchestrates, one file draws charts, one shell renders cards, one theme defines the visual system.

Frontend Deep Dive: Reports Cleanup

Preventing duplicate mental models

Removing AnalyticsGrid from ReportsLanding.jsx prevents the same chart experience from living in two places.

This is important for maintainability because future chart improvements should target /analytics as the single dashboard destination.

D: Frontend Rule

One page orchestrates, one file draws charts, one shell renders cards, one theme defines the visual system.

Verification Summary

Build and smoke status

The implementation transcript reports successful endpoint tests for the four new charts, a clean frontend production build, Phase 10 smoke passing at 49/49, and Phase 11 smoke passing at 23/23.

This means the Advanced Analytics change did not regress the sealed PDF generation phase.

Verification	Result
New endpoints	weekly-activity, jc-lifecycle-funnel, equipment-registration-trend, priority-mix-trend all returned 200
FE build	Vite build completed successfully
Phase 10 smoke	49 passed, 0 failed
Phase 11 smoke	23 passed, 0 failed

RBAC Matrix

Who can access analytics and exports

Analytics page visibility depends on analytics:view. Backend chart endpoints accept analytics:view or reports:view-analytics. CSV buttons are shown only when reports:export is present.

Role	Analytics Page	Chart JSON	CSV Export Button
SUPER_ADMIN	Allowed	Allowed	Allowed if reports:export present
LAB_IN_CHARGE	Allowed	Allowed	Allowed if reports:export present
LAB_ENGINEER	Allowed	Allowed	Role permission dependent
NORMAL_USER	Allowed by Part II grant	Allowed	Usually limited by export permission
VIEW_ONLY	Allowed by Part II grant	Allowed	Hidden if no reports:export

Risk Register

Advanced Analytics risks and controls

The biggest risks are not code syntax risks. The biggest risks are semantic chart misuse, endpoint drift, dashboard performance, over-refreshing, and UI clutter.

Risk	Impact	Control
Wrong aggregation meaning	User may make wrong decision	Document each chart definition and SQL source
JSON/CSV mismatch	Audit distrust	Use same data path and smoke test both
Dashboard too busy	Reduced usability	Spacious layout and grouped rows
Excess polling load	Backend pressure	30s interval and lightweight aggregate queries
Duplicate analytics in Reports	Maintenance confusion	Remove AnalyticsGrid from Reports page

Performance Reasoning

Why the dashboard remains practical

Each chart endpoint returns aggregate rows, not full operational records. Monthly and weekly buckets are compact. This keeps the 30-second auto-refresh realistic.

If production grows significantly, the next optimization is not to rewrite the UI but to add indexes, materialized views, or cached chart read models.

Deployment Notes

What to watch during handoff

After deployment, verify that /reports no longer contains the chart grid, /analytics is visible in the sidebar for allowed roles, chart data loads under real authentication, and CSV download buttons obey reports:export.

Also confirm the dashboard remains readable at standard engineering monitor sizes.

Future Enhancement Roadmap

What can come after Part III

Possible future work includes drill-down from chart to report table, server-side caching, saved analytics views, anomaly markers, threshold alerts, PDF export of the analytics dashboard, and role-specific default layouts.

These are future enhancements. They are not required for the current sealed Advanced Analytics phase.

Appendix A - Backend Endpoint Smoke Test

Light code appendix - documentation excerpt

This appendix preserves the implementation intent without turning the document into a raw source dump.

The code is shown to anchor the engineering decision, not to replace the repository.

Code excerpt

```
for (const p of ['weekly-activity', 'jc-lifecycle-funnel', 'equipment-registration-trend', 'priority-mix-trend']) {  
  const r2 = await fetch('http://localhost:3000/api/v1/analytics/' + p + '?months=6', {  
    headers:{Authorization:'Bearer '+token}  
  });  
  const j = await r2.json();  
  console.log(p.padEnd(32), r2.status, 'rows=', Array.isArray(j.data) ? j.data.length : '?');  
}
```

Appendix B - Chart Theme Palette

Light code appendix - documentation excerpt

This appendix preserves the implementation intent without turning the document into a raw source dump.

The code is shown to anchor the engineering decision, not to replace the repository.

Code excerpt

```
export const PALETTE = [  
  '#3b82f6', '#10b981', '#f59e0b', '#ef4444',  
  '#8b5cf6', '#06b6d4', '#f97316', '#14b8a6'  
];  
export const ANIMATION_MS = 700;  
export const GRID = { strokeDasharray: '3 3', stroke: '#e5e7eb', vertical: false };
```

Appendix C - ChartCard Contract

Light code appendix - documentation excerpt

This appendix preserves the implementation intent without turning the document into a raw source dump.

The code is shown to anchor the engineering decision, not to replace the repository.

Code excerpt

```
export function ChartCard({
  title, subtitle, stat, height = 320,
  loading, error, isFetching, dataUpdatedAt,
  onRefresh, onDownloadCsv, span = 1, children,
}) {
  // Reusable spacious shell for every analytics card.
}
```

Appendix D - Analytics Refresh Contract

Light code appendix - documentation excerpt

This appendix preserves the implementation intent without turning the document into a raw source dump.

The code is shown to anchor the engineering decision, not to replace the repository.

Code excerpt

```
const DEFAULT_POLL_MS = 30_000;
const params = useMemo(() => ({ months, divisionId: divisionId || undefined }), [months, divisionId]);
function refreshAll() {
  qc.invalidateQueries({ queryKey: ['chart'] });
  toast.message('Refreshing all 12 charts...');
}
```

Appendix E - G9 Weekly Activity Component Pattern

Light code appendix - documentation excerpt

This appendix preserves the implementation intent without turning the document into a raw source dump.

The code is shown to anchor the engineering decision, not to replace the repository.

Code excerpt

```
export function WeeklyActivity({ params, pollMs }) {  
  const q = useChartQ('weeklyActivity', params, pollMs);  
  const data = q.data || [];  
  return <ChartCard title='Weekly Activity Trend' span={2}>...</ChartCard>;  
}
```

Appendix F - CSV Pairing Rule

Light code appendix - documentation excerpt

This appendix preserves the implementation intent without turning the document into a raw source dump.

The code is shown to anchor the engineering decision, not to replace the repository.

Code excerpt

```
function downloadHandler(key, params, title) {  
  return async () => {  
    const { filename } = await downloadChartCsv(key, params);  
    toast.success(`Downloaded ${filename}`);  
  };  
}
```

Appendix G - Reports Cleanup

Light code appendix - documentation excerpt

This appendix preserves the implementation intent without turning the document into a raw source dump.

The code is shown to anchor the engineering decision, not to replace the repository.

Code excerpt

```
// ReportsLanding.jsx  
// AnalyticsGrid import and analytics section were removed.  
// Reports now focuses on report cards, filters, tables, PDF/CSV export.
```

Appendix H - CHART_PATHS Extension

Light code appendix - documentation excerpt

This appendix preserves the implementation intent without turning the document into a raw source dump.

The code is shown to anchor the engineering decision, not to replace the repository.

Code excerpt

```
weeklyActivity: 'weekly-activity',  
jclifecycleFunnel: 'jc-lifecycle-funnel',  
equipmentRegistrationTrend: 'equipment-registration-trend',  
priorityMixTrend: 'priority-mix-trend',
```

Appendix I - Funnel Data Shape

Light code appendix - documentation excerpt

This appendix preserves the implementation intent without turning the document into a raw source dump.

The code is shown to anchor the engineering decision, not to replace the repository.

Code excerpt

```
const funnelData = data.map((r) => ({  
  name: r.stage,  
  value: r.count,  
  fill: STATUS_COLORS[r.stage] || PALETTE[0],  
}));
```

Appendix J - Priority Mix Visual Contract

Light code appendix - documentation excerpt

This appendix preserves the implementation intent without turning the document into a raw source dump.

The code is shown to anchor the engineering decision, not to replace the repository.

Code excerpt

```
<Area type='monotone' dataKey='low' stackId='p' />  
<Area type='monotone' dataKey='medium' stackId='p' />  
<Area type='monotone' dataKey='high' stackId='p' />
```

Appendix K - Smoke Regression Result

Light code appendix - documentation excerpt

This appendix preserves the implementation intent without turning the document into a raw source dump.

The code is shown to anchor the engineering decision, not to replace the repository.

Code excerpt

```
Phase 10 (charts): 49 passed, 0 failed
Phase 11 (PDF):   23 passed, 0 failed
FE production:    vite build completed successfully
```

Appendix L - Final Memory Note

Light code appendix - documentation excerpt

This appendix preserves the implementation intent without turning the document into a raw source dump.

The code is shown to anchor the engineering decision, not to replace the repository.

Code excerpt

```
The /analytics page now feels like a live ops dashboard: wavy curves breathing,  
badges pulsing, charts auto-refreshing every 30 seconds, click-to-refresh anywhere,  
CSV-export anywhere. The /reports page is leaner and focused on its core PDF-export job.
```

Source Transcript Audit

What the uploaded text contributed

The uploaded transcript was used as the authoritative source for implementation events, file changes, endpoint verification, smoke results, and final delivered summary.

Audit Item	Count / Value
Uploaded file lines	1262
Mentioned new endpoints	20
Mentions of smoke	17
Mentions of build	12

Final Locked Seal

Phase 10 Part III - Advanced Analytics

Phase 10 Part III is sealed as the Advanced Analytics module. It separates Reports from Analytics, adds four new advanced chart endpoints, redesigns the analytics dashboard into twelve chart cards, keeps CSV exports, preserves RBAC, and verifies with green smoke tests.

This document should be treated as the final technical explanation for the phase unless the project explicitly reopens Phase 10 Part III.

LOCKED: Advanced Analytics Seal

Phase 10 Part III is accepted as completed and documented in DS senior-engineering format.

CONTROL: Regression Seal

Phase 10 smoke: 49/49 green. Phase 11 smoke: 23/23 green. FE build: clean. No regression reported in the transcript.